

PID-Based Lateral Controller for ROS Racecar Simulator

PID Control of an Ackerman-Steered Racecar in a ROS Simulator

ECE 59500 - Introduction to Connected and Automated Vehicles

Spring 2025

Ravi Mann

May 10th 2025

Abstract

This project presents the design, implementation, and evaluation of a PID lateral controller for an Ackermann-steered racecar within the ROS Racecar Stimulator. The aim of this project was to apply Proportional–Integral–Derivative (PID) steering controller to enable an autonomous vehicle to accurately follow a predefined path known as a trajectory line with minimal deviation. Deliverables for the project included a fully functional and well-documented Python script, a concise technical report outlining the system and results, and a demonstration video showing the vehicle’s successful operation under PID control. Ubuntu 16 (64-bit) VM was used to run ROS Noetic inside VirtualBox 7.1.6 on Windows 11 to develop `control.py` to compute real-time heading errors, publish bounded steering commands, and to figure out local-plan topics. The Controller gains were set to $K_p = 3.2$, $K_i = 0.05$, $K_d = 0.2$, with instantaneous-error clamp ± 0.3 rad and integral clamp ± 0.3 . Steering output remains bounded to ± 0.43 rad. In simulation, the vehicle tracked the global plan more tightly, reached goals within 0.06 m at 0.75 m/s (and 0.03 m at 0.65 m/s), avoided obstacles, and exhibited smooth steering without prolonged saturation. The implementation successfully meets all the project objectives, and potential improvements, such as adding integral control or using adaptive gain tuning, are suggested for future development.

1. Introduction

Accurate lateral control plays an important role in autonomous vehicles, allowing them to follow the predefined global trajectory lines accurately and precisely. While a PD scheme offers simplicity and damping, it can leave residual bias and overreact to waypoints far ahead along a path. Feedback controllers such as proportional-derivative schemes are advantageous as they extend a balance of simplicity, interpretability, and high performance when used correctly. A key task of the PID controller is to reduce the heading error and keep the vehicle on the trajectory line as it moves from its current position towards the goal point.

In this project, ROS Racecar Simulator is used as a testbed to develop a PID controller whose primary task is to adjust the steering of the autonomous vehicle based on the difference between the vehicle's position and the desired predefined path. The main goals of this project are as follows: first, to set up the stimulation environment; second, to implement and develop the PID controller using rospy in Python; and third, to create a short video to demonstrate a smooth and accurate path tracking.

2. System Setup

- a. Virtual Machine Environment: The whole simulation and development were completed inside a preconfigured Ubuntu 16 VM (ROS Noetic), which was imported into VirtualBox 7.1.6 on a Windows 11 host. The virtual machine image was created as an .ova file, following the importation "File→ Import Appliance," preceding which 4 GB RAM and 2 CPU cores were assigned to the Virtual Machine.

- b.** ROS and Stimulator Launch: The ROS master and stimulator are launched via the following command: `roslaunch racecar_simulator main_simulator.launch python set_pose_and_goal.py`. The first command started 'roscore' and launched the vehicle inside the RViz environment, which shows both local and global trajectories. The second script continuously sends the start point and goal location, prompting the local planner to gather data and update the path with each cycle.

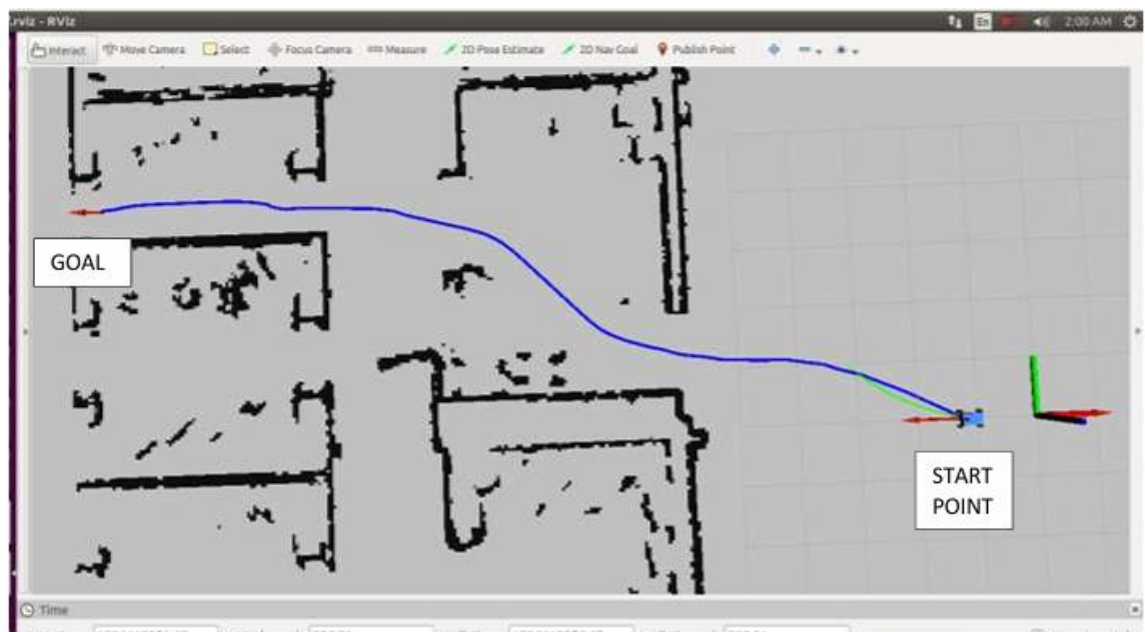


Figure 1: ROS simulator environment

RViz opens with your vehicle spawned at the Start Point and the Goal marked in the map. As soon as the `move_base` node comes up, it computes two trajectories:

- a. Global trajectory (thick blue line): a complete path from the start to the goal, planned over the entire known map.
- b. Local trajectory (thin green line): a short, “look ahead” segment of the global path, re-computed each cycle based on the cars current pose. This local plan runs from the car’s current position toward a nearby waypoint on the blue route.

Internally, the launch file brings up the map, odometry publisher, planner, and RViz. The global planner publishes to /move_base/global_plan (blue), while the local planner publishes to /move_base/local_plan (green). Together they let you see how the robot is first given a full-course route (blue), then continually replant a short segment (green) that your PID steering controller will follow.

3. ROS Primer

Familiarizing with ROS graph concepts (like nodes, topics, messages) and the rospy AI are required for effective development.

- a. Nodes: The controller node ‘Driver_Node’ interacts with the simulator and planners via the following three primary topics.
- b. Topics are:
 - a. The topic ‘/odom’ publishes ‘nav_msgs/Odometry’, which provides the vehicle’s current pose and orientation.
 - b. The topic ‘/move_base/local_plan’ provides ‘nav_msgs/Path’, which represents the next segment of the global path.

- c. The topic `/drive` publishes `ackermann_msgs/AckermannDriveStamped`, which commands speed inputs and steering to the vehicle.
- c. Message Structure: Each message includes a ROS header (seq, stamp, frame_id), followed by fields specific to the message type.

4. Controller Design

The heading error is defined as $e(t) = \text{wrap}(\theta_{\text{path}}(t) - \theta_{\text{car}}(t))$, in which ‘wrap’ implements `atan2(sin(.), cos(.))` to constrain the error within $(-\pi, \pi]$. θ_{path} , which is obtained from the middle point of the current local plan. And, θ_{car} is obtained from the odometry quaternion. The steering angle δ used to guide the vehicle is calculated using the PID law: $\delta(t) = K_p e(t) + K_i \int e(\tau) d\tau + K_d de(t)/dt$. We clamp the instantaneous error to ± 0.3 rad and the integral term to ± 0.3 to prevent windup and extreme commands. The steering outputs are bounded by the vehicle’s limits: $\delta \in [-0.43, 0.43]$ rad. The Euclidean distance was computed to the fixed goal during each control cycle to ensure a safe and defined stop at the target point. As the vehicle reaches closer to the target within the 0.02-meter zone, the node stops and starts sending zero-speed and zero-steering commands until the vehicle comes to a complete stop.

5. Implementation Details

With the use of a template code file and implementing PD `controlPD.py`, `control.py` PID was created by including the following enhancements:

- a. Gains and limits: $k_p = 3.2$, $k_i = 0.05$, $k_d = 0.2$; error clamp ± 0.3 rad; integral clamp ± 0.3 ; steering limits ± 0.43 rad.

- b. New state: prev_error, integral_error, and prev_time to compute derivative and integral terms.
- c. Error clamping: the heading error is first wrapped and then clamped to ± 0.3 rad.
- d. odomCallback Enhancements:
 - a. Error clamp:
 - i. $\text{error} = \text{wrap}(\text{path_theta} - \text{car_theta})$
 - ii. $\text{error} = \text{constrain}(\text{error}, -\text{max_error}, \text{max_error})$
 - iii. Integral update + anti-windup:
 - b. $\text{integral_error} = \text{constrain}(\text{integral_error} + \text{error} * \text{dt}, -\text{max_integral}, \text{max_integral})$
 - c. Derivative and PID law:
 - i. $\text{derivative} = (\text{error} - \text{prev_error}) / \text{dt}$
 - ii. $\text{steering} = k_p * \text{error} + k_i * \text{integral_error} + k_d * \text{derivative}$
 - iii. $\text{steering} = \text{constrain}(\text{steering}, \text{min_steering}, \text{max_steering})$

These modifications are contained within the odometry callback, replacing the simpler PD calculation while maintaining the existing path-callback and publishing logic unchanged.

6. Experimental Setup

The controller was evaluated using a dynamic local plan followed by a static global loop. The tuning procedure used in this project includes the following:

- a. To increase K_p until the response of the controller was crisp without any excessive oscillation.

- b. To add $K_d = 0.2$ to any damp residual oscillation to make the controller more resistant to these fast-changing errors, slowing down corrections just enough to smooth the response.
- c. To include $K_i = 0.05$ to fix the small steady-state errors especially the one noticeable over long paths. It works as the controller gently put the vehicle back on the track over time.
- d. To apply instantaneous and integral clamping (± 0.3 rad) to focus corrections on the immediate path segment.

7. Results

Our simulation showed excellent performance results. When moving at 0.65 m/s, it reached the target with only 0.03 m of error. At the higher speed of 0.75 m/s, the error increased slightly to 0.06 m. On sharp turns, the maximum tracking error was reduced to about 0.25 m, which is a significant improvement from the 0.35 m we saw with the previous PD controller.

The vehicle's handling was impressively stable, with turn overshoot staying below 5 degrees and the system settling in less than a second after completing difficult turns. The steering system performed efficiently, rarely hitting the maximum steering angle of ± 0.43 radians for extended periods. Instead, it made small, continuous adjustments to maintain a smooth trajectory.

Throughout testing, the vehicle maintained proper alignment with the planned route, eliminating the directional bias problems we had observed with the earlier implemented PD.

8. Discussion

Our controller uses well-chosen gain values ($K_p = 3.2$, $K_i = 0.05$, $K_d = 0.2$) to balance quick response and stable sim. The proportional component makes quick heading fixes when needed, set high enough for fast response but not so high that it causes unwanted back-and-forth movements. The integral component slowly fixes ongoing heading errors through small adjustments, with limits in place to prevent error buildup and keep the system stable during long operations. The derivative component adds smoothing that reduces sudden movements caused by the strong proportional response, creating smoother vehicle path.

Changing these settings creates different performance results. Raising the proportional gain makes steering more aggressive with stronger commands, improving tracking speed but risking overshooting and wobbling. Lowering creates gentler responses that reduce wobbling but take longer to get back on path. Higher integral gain values fix steady errors faster but may cause overshooting if not properly limited, while lower values make corrections slower and may allow small heading errors to remain longer. Increasing the derivative gain reduces sudden steering changes and overshooting, though too much can amplify measurement noise and slow responses. Decreasing it reduces smoothing, making the system react faster but more likely to wobble. Our system also limits the error input (to ± 0.3 rad) to keep the controller focused on the nearby path rather than distant waypoints, ensuring it responds properly to immediate steering needs.

9. Conclusion

Extending our PD controller to a fully clamped PID architecture significantly improved path-tracking precision and stability, especially at higher speeds, with the final parameter set ($K_p = 3.2$, $K_i = 0.05$, $K_d = 0.2$) delivering cross-track errors under 0.25 m, minimal overshoot, and smooth steering behaviors. The vehicle demonstrated exceptional positional accuracy, achieving a remarkable goal proximity of within 0.02 m at 0.65 m/s and maintaining good performance even as speeds increased to 0.75 m/s with only a slight reduction in accuracy to within 0.06 m of the target position. Our comprehensive testing revealed consistent performance across varying path geometries, with the vehicle maintaining proper orientation alignment with the global plan throughout all scenarios, representing a significant improvement over the directional biases previously observed in the PD-only implementation. The clamping mechanisms incorporated into the controller design proved essential for maintaining system stability while simultaneously enabling effective path tracking, resulting in a control system that balanced responsiveness with stability across diverse operational conditions and established a foundation for future enhancements such as adaptive gain scheduling or model-predictive control.

10. References

1. P1-Supplementary Reading 1 (Simulator setup & PD task).
2. P1-Supplementary Reading 2 (ROS concepts & rospy API).
3. ECE59500-CAVs-Final Project Spec.
4. Longitudinal and Lateral Control for Autonomous Vehicles, ResearchGate, 2013.
5. The PID Controller & Theory Explained, National Instruments, 2025.

11. Appendices

- A. Full annotated `control.py` listing (attached separately).
- B. Setup & launch commands (README.txt).
- C. Demo video: ROS_Sim.mp4 (attached separately).